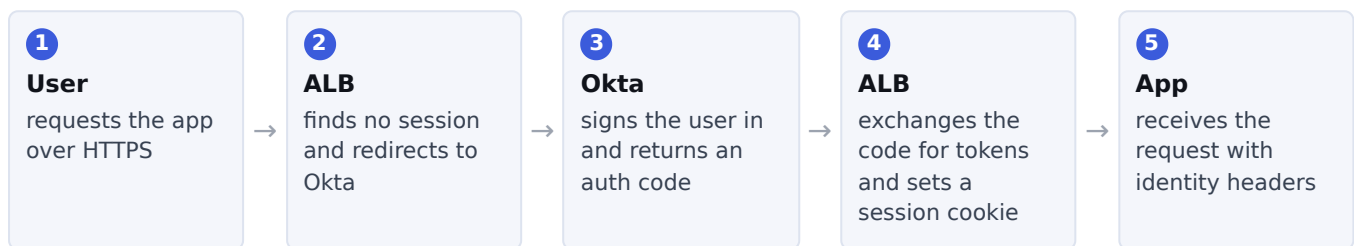


Integrating AWS ALB with Okta

Okta sign-in handled by the load balancer's built-in OIDC support. Your app needs no auth code.

AWS Application Load Balancer has built-in OIDC authentication, so it can sign users in with Okta directly. The ALB handles the login redirect, the token exchange and the session cookie, then passes the user's identity to your app as request headers.

How it works



Step 1: Create the app in Okta

Okta Admin → Applications → Create App Integration

- Sign-in method: **OIDC - OpenID Connect**
- Application type: **Web Application**
- Grant type: **Authorization Code**
- **Sign-in redirect URI:** `https://<your-app-domain>/oauth2/idresponse`

The path `/oauth2/idresponse` is fixed by ALB and has to match exactly. Use the domain your users type in the browser, not the ALB's `*.elb.amazonaws.com` name.

Save the app, then note the **Client ID** and **Client Secret**, and assign the users or groups who should have access.

Step 2: Collect the Okta endpoints

If you use the `default` custom authorization server:

ALB FIELD	VALUE
Issuer	<code>https://<okta-domain>/oauth2/default</code>
Authorization endpoint	<code>https://<okta-domain>/oauth2/default/v1/authorize</code>
Token endpoint	<code>https://<okta-domain>/oauth2/default/v1/token</code>
User info endpoint	<code>https://<okta-domain>/oauth2/default/v1/userinfo</code>

If you use the org authorization server instead, drop `/default` from the paths. The issuer then becomes just `https://<okta-domain>`. To confirm the exact values, open `https://<okta-domain>/oauth2/default/.well-known/openid-configuration`.

Step 3: Configure the ALB listener

The listener **must be HTTPS (443)** with an ACM certificate, because OIDC auth isn't available on HTTP listeners.

EC2 → Load Balancers → Listeners → HTTPS:443 → Edit rules, then add two actions in this order:

1. **Authenticate → OIDC**: enter the issuer, endpoints, client ID and secret from Steps 1 and 2
2. **Forward** → your target group

Terraform version

```
resource "aws_lb_listener_rule" "okta_auth" {
  listener_arn = aws_lb_listener.https.arn
  priority     = 10

  action {
    type = "authenticate-oidc"
    order = 1

    authenticate_oidc {
      issuer              = "https://your-org.okta.com/oauth2/default"
      authorization_endpoint = "https://your-org.okta.com/oauth2/default/v1/authorize"
      token_endpoint       = "https://your-org.okta.com/oauth2/default/v1/token"
      user_info_endpoint   = "https://your-org.okta.com/oauth2/default/v1/userinfo"
      client_id            = var.okta_client_id
      client_secret         = var.okta_client_secret
      scope                 = "openid profile email"
      session_timeout       = 3600 # seconds; ALB default is 7 days
      on_unauthenticated_request = "authenticate"
    }
  }

  action {
    type      = "forward"
    order     = 2
    target_group_arn = aws_lb_target_group.app.arn
  }

  condition {
    path_pattern { values = ["/*"] }
  }
}
```

`on_unauthenticated_request` accepts three values:

<code>authenticate</code>	Redirects to Okta. Use this for browser apps.
<code>deny</code>	Returns a 401. Use this for APIs, where a redirect would break clients.
<code>allow</code>	Lets the request through without a login. Use this for pages where login is optional.

Step 4: Allow the ALB to reach Okta

The ALB calls Okta's token and user info endpoints itself, from inside your VPC. That means **the ALB's security group needs outbound HTTPS (443) to the internet**. An internal ALB also needs a NAT gateway.

This is the most common reason setups fail. The Okta login page works, and then the user gets a **500 error** on the redirect back, because the ALB can't reach Okta to exchange the code.

Step 5: Read the user in your app

After login, the ALB adds three headers to every request it forwards:

HEADER	CONTENTS
<code>x-amzn-oidc-identity</code>	The user's subject ID (<code>sub</code>)
<code>x-amzn-oidc-data</code>	A JWT with the user's claims (email, name, etc.), signed by the ALB
<code>x-amzn-oidc-accesstoken</code>	The access token from Okta

Verify the signature before trusting `x-amzn-oidc-data`

```
import jwt, requests

REGION = "ap-south-1"

def get_user(headers):
    token = headers["x-amzn-oidc-data"]
    kid = jwt.get_unverified_header(token)["kid"]
    pub_key = requests.get(
        f"https://public-keys.auth.elb.{REGION}.amazonaws.com/{kid}"
    ).text
    return jwt.decode(token, pub_key, algorithms=["ES256"]) # ALB signs with ES256

# claims = get_user(request.headers) -> {"sub": ..., "email": ..., "name": ...}
```

Gotchas

Accept traffic only from the ALB. The backend's security group should allow inbound traffic from the ALB's security group alone. Otherwise anyone who can reach the backend directly can send fake `x-amzn-oidc-*` headers.

Logout isn't automatic. ALB has no logout endpoint. To log a user out, expire the `AWSELBAuthSessionCookie-*` cookies (set their expiry in the past), then redirect to Okta's `/v1/logout`. If you only clear the cookie, the Okta session is still active and the user is signed straight back in.

The default session is 7 days. Set `session_timeout` to match your security policy.

Groups aren't included by default. To get Okta groups in the claims, add a `groups` claim on the authorization server (**Security** → **API** → **default** → **Claims**) and add `groups` to the ALB's `scope`.